Assignment 4: PPO Implementation and Performance Comparison with REINFORCE, Actor Critic, Advantage Actor Critic and DQN in the CartPole-v1 Environment

Santiago Moreno Mercado
s4614151

Abstract

Reinforcement learning is an area of machine learning that focuses on the development of agents. These are able to make decisions, selecting actions that affect the environment it is in, and receive rewards based on the quality of those actions. This report focuses on an implementation of PPO, a state-of-the-art reinforcement learning algorithm, together with experiments that were performed using this implementation. A limited amount of hyperparameter tuning was done, and the resulting learning curve is compared with learning curves of other fundamental reinforcement learning algorithms, including DQN, REINFORCE, AC and A2C, of which the implementations and results were gathered in previous assignments. Through the experiments, it was found that in the CartPole-v1 environment PPO outperformed all other agents, converging faster while maintaining stable performance throughout most of the final experiments.

1. Introduction

Reinforcement learning is a branch of machine learning that trains an agent to maximize reward within an environment, by taking actions that transition to different states.(Schulman et al., 2017) introduced Proximal Policy Optimization (PPO) in 2017, and since then it has become one of the widely used algorithms in reinforcement learning, because of both its results and stability. The method that is implemented in this assignment utilizes clipping, a concept inspired by Trust Region Policy Optimization (Schulman et al., 2015), which estimates the expected performance improvement of the algorithm, and limits the possible policy updates to be within an estimated interval, making it stable and reliable when compared to other model-free policy methods such as vanilla Actor Critic (AC).

In this work, the focus is the clipping version of PPO. How it differs from other methods, and we compare this implementation of PPO with fundamental methods in reinforcement learning like AC, Advantage Actor Critic(A2C), REINFORCE and DQN, all in the CartPole-v1 environment. [1]

2. Theory

The objective of a Reinforcement Learning algorithm is to find the optimal policy $\pi_\theta(a|s)$, where given a state s, it outputs the probability of selecting an action a, with the purpose of maximizing the expected return $Q(s, a)$.

For Deep Reinforcement Learning algorithms, neural networks are used to approximate this action selection policy $\pi_\theta(a|s)$ optimization process, where θ represents the parameters that describe its behaviour. In AC-style methods, this first network is known as the actor, but they also add a counterpart defined as the critic $V_\phi(s)$ with parameters ϕ, which helps guide the actor towards the optimal policy.

These two networks use different loss functions to update their parameters, and ,in turn, the policy. For the critic, the loss function is based on the squared difference between the estimated critic state values and the rewards:

$$L_V(\phi) = \mathbb{E}_t[(Q_\phi(s_t, a_t) - R_t)^2] \tag{1}$$

While the vanilla actor loss function uses the log probabilities of the actor policy, as well as the critic state values:

$$L(\theta) = -\mathbb{E}_t[\log \pi_\theta(a_t|s_t) Q_\phi(s_t, a_t)] \tag{2}$$

A2C improves over this last loss function by using the advantage function, which quantifies the advantage of taking an action a compared to the expected outcome of the policy. This reduces variance and adds stability, and results in this new actor loss function:

$$L_{A2C}(\theta) = -\mathbb{E}_t[\log \pi_\theta(a_t|s_t) \hat{A}(s_t, a_t)] \tag{3}$$

In this implementation, normalized advantages were used to increase stability further.

[1] Cart Pole - Gymnasium Documentation

Proximal Policy Optimization (PPO) takes the A2C loss function as a starting point, and makes key improvements to increase stability and to avoid collapsing performance.

2.1. Clipped Surrogate Loss

The loss used for PPO comes from the fundamental idea of updating the policy in measured steps, avoiding overshooting the optimum, but still making significant progress and exploration. Previous works like Trust Region Policy Optimization (TRPO) used the policy update scheme introduced by Kakade and Langford(Kakade & Langford, 2002), minimizing its loss within the constraints of KL divergence which served as guidance for defining the size of each step to take and fixing the overshooting problem, but was too mathematically complex, causing update rules to be hard to implement in a computationally efficient and practical way.

PPO takes inspiration from TRPO, but instead of defining the trust region with KL divergence, PPO creates bounds using clipping:

$$L_{CLIP}(\theta) = -\mathbb{E}_t[\min(r_t(\theta)\hat{A}(s_t, a_t), \\ clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}(s_t, a_t))] \tag{4}$$

$r_t(\theta)$ represents the ratio of the old policy and the new policy, given by:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{5}$$

Which are computed within the code using log probabilities for stability:

$$logr_t(\theta) = log\pi_\theta(a_t|s_t) - log\pi_{\theta_{old}}(a_t|s_t) \tag{6}$$

And ϵ is a hyperparameter that defines the extent of the bounds, and as it increases larger jumps in policies are allowed and vice-versa. What this new loss does is it uses the ratios between the new and old policies directly instead of just doing changes at a parameter level, avoiding the issue where small parameter changes can cause significant instability in the policy, and it also removes the incentive to move r_t outside of the interval defined by the ϵ parameter $[1-\epsilon, 1+\epsilon]$. This effectively defines a trust region for new policy updates, improving stability by making steps only within these bounds.

This loss can be improved by adding an entropy bonus term S gathered from the new action distribution, which increases exploration. The full actor loss function is now defined as:

$$L_{ACT}(\theta) = L_{CLIP}(\theta) - cS[\pi_\theta](s_t) \tag{7}$$

Where c is a hyperparameter that dictates the impact of the entropy bonus.

Besides the loss, other components and modifications to the algorithm have been added in this implementation, with the objective of increasing stability further.

2.2. Increasing Stability

When it comes to calculating the estimated cumulative reward, methods like REINFORCE and the AC implementation for the last assignment used Monte Carlo value estimates:

$$\hat{Q}_{MC}(s_t, a_t) = \sum_{k=0}^{T-t} \gamma^k r_{t+k} \tag{8}$$

Where T is the number of rewards in one trajectory τ, and γ is the discount factor used to reduce distant rewards. The method can yield good results, and has low bias, but has high variance as well, as relying on a single trajectory can introduce significant noise.

To deal with this variance, in this implementation rollout batches were used instead, with which multiple trajectories are sampled, until a total of Z steps are taken.

$$\hat{Q}_{ROLL}(s_t, a_t) = \sum_{k=0}^{Z-t} \gamma^k r_{t+k} \tag{9}$$

This operates similarly to MC estimation but makes it more stable by sampling more than one single trace for the gradient updates, increasing the quality of the policy updates.

To further enhance this, the rollout batch gets split into K minibatches,and the loss is then calculated and backpropagated for the actor and critic individually, the critic using equation 1 and the actor using equation 7. Minibatching breaks down what would be a massive high variance policy update into smaller updates, reducing the chances of overshooting the optimum and in turn increasing stability. This policy update process is then repeated for M epochs.

Two more additions were made to deal with long executions, where these multiple M update repetitions could cause instability or collapse. One of them is Approximate KL Divergence early stopping, where the difference between the old policy $\pi_{\theta_{old}}$ and the new policy π_θ is evaluated between epochs:

$$KL(\theta) = (r_t(\theta) - 1) - logr_t(\theta) \tag{10}$$

Where $r(\theta)$ is the ratio between the policies explained in 5. This is an unbiased estimator explained by Schulman in (Schulman, 2020), helping to preserve low variance. If this estimator goes over a threshold kl, the next epoch is stopped and a new rollout gets sampled. The last addition is gradient clipping on the actor parameters, which is used to avoid exploding gradients, as these can become too large during back propagation. It calculates the norm of the gradient, and scales it down if it goes over a norm threshold.

Algorithm 1 PPO Algorithm

Input: Number of timesteps N, discount factor γ, learning rate α, Number of minibatches K, Number of epochs M, Loss bounds ϵ, Entropy coefficient c, Target kl
Initialize policy $\pi_\theta(a|s)$ with random weights θ and value model $V_\phi(s)$ with random weights θ and ϕ
repeat
  Sample rollout $T = \{\tau_0, \tau_1, \ldots, \tau_n\}$ following $\pi_\theta(a|s)$, such that the sum of the timesteps over the rollout equals Z
  $R_t = \sum_{k=0}^{Z-t} \gamma^k r_{t+k}$
  $\hat{A}_t = R_t - V_\phi(s_t)$
  for $i = 1, ..., M$ **do**
    for $j = 1, ..., K$ **do**
      $KL(\theta) = (r_t(\theta) - 1) - logr_t(\theta)$
      Backpropragate actor loss $L_{ACT}(\theta)$ and critic loss $L_V(\phi)$, using gradient clipping and learning rate α_θ for the actor, and learning rate α_ϕ for the critic.
      if $KL > kl$ **then**
        break
      end if
    end for
    if $KL > kl$ **then**
      break
    end if
  end for
until N timesteps passed

Algorithm 1 shows the final PPO implementation used for this assignment, starting with the initialization of the policy and value models $\pi_\theta(a|s)$, $V_\phi(s)$, followed by the sampling of the rollout batch T including multiple episode traces τ. Then both the return R_t and advantages $\hat{A}_t$, which is normalized but it was not shown in the algorithm for readability, are calculated before starting the policy update epochs, and from then backpropagation of both the actor loss $L_{ACT}(\theta)$, which is made with gradient clipping, and the critic loss $L_V(\phi)$, while checking every minibatch for the KL divergence early stop condition, in case it ever goes over the hyperparameter kl.

3. Experiments

3.1. Setup

The environment used in the experiments is the v1 version of the CartPole implementation provided by the gymnasium library (Towers et al., 2024). It focuses on balancing a pole on a moving cart. The environment simulates gravity acting on the pole, and the goal is to apply forces to the left or right on the cart to try and keep the pendulum upright, as the cart moves in a frictionless track. The state space of the environment consists of four continuous values, which are the cart position, cart velocity, pole angle, and pole angular velocity. The action space consists of two actions: moving left (0) and moving right (1). On every timestep, if the pole is still upright, a reward of 1 is given. The episode ends if the pole falls or after 500 steps.

The policy and value networks are implemented as separate networks, each with one input layer, two hidden layers with ReLU activation functions, and one output layer, and both networks use hidden layers with 128 units each. The policy network's output layer applies a Softmax function to get a probability distribution of the possible actions, while the value network outputs one value estimate. The optimizer was Adam, with a fixed learning rate of 10^{-4} for both networks. Updates are performed on sampled rollouts of 2000 timesteps, which are divided into K minibatches of size B, repeating these updates for 4 epochs. The algorithm was implemented as Algorithm 1, with a discount factor $\gamma = 0.99$ and a budget of timesteps $N = 10^6$, using normalized advantage $\hat{A}$ to increase stability.

The agent was evaluated after every 250 steps for 3 episodes, and the result was averaged and for later analysis. The final plot comes from the averaging of 5 repetitions of the algorithm being trained through the whole budget to get a more accurate result and information regarding the deviation across different runs. Taking into account all of the hyperparameters, only 4 were used for hyperparameter testing, these were the value of the clipping parameter ϵ, the entropy coefficient c,the target KL Divergence value kl and the minibatch length or size B. The hyperparameter testing was done with 3 different values for the first 3 parameters and 2 for B.

When it comes to the DQN, REINFORCE, AC and A2C results, the returns were retrieved from the previous assignments, all of which had the same evaluation interval of every 250 steps with the same budget of 10^6 timesteps and they were all also averaged over 5 repetitions. All of the algorithms had a learning rate of $\alpha = 10^{-4}$, except for the DQN with a learning rate of $\alpha = 6.25 \times 10^{-5}$ and the AC critic network with a learning rate of $\alpha = 10^{-3}$.

3.2. Results

Table 1 shows the selected hyperparameters for the final plot. The hyperparameter tuning process was not as extensive as it could have been due to time constraints and issues with initial versions of the algorithm. Additional hyperparameter testing information can be found in the appendix.

4. Discussion

The experiments show how PPO outperformed the other agents with respect to most metrics. It showed how the implementation of rollout batches, minibatching ,learning

Hyperparameter	ϵ	c	B	kl
Value	0.05	0.01	50	0.01

Table 1. Selected hyperparameter values for PPO after the testing process. These were collected by running every combination of 3(ϵ, c, kl) or 2(B) parameter values, running it with 2×10^5 timesteps as budget and averaging over 2 runs for each combination, plotting all of the results and selecting those that reached high rewards quickly while staying stable for final comparisons before selecting the final set of parameters.

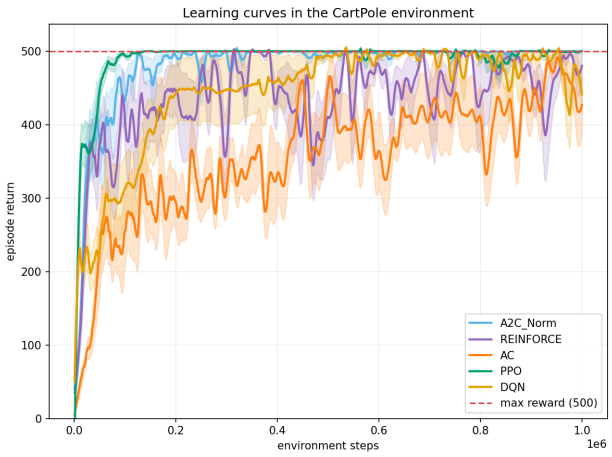


Figure 1. Learning Curves for PPO, normalized Advantage Actor-Critic, Actor-Critic, REINFORCE, and DQN. All results are averaged over 5 runs, with the shaded area showing the standard deviation of the runs. A Savitzky-Golay filter with a smoothing window of 81 was applied, which at times shows when the curve goes over the maximum possible reward of 500. From the results, PPO is both the fastest in reaching the max reward threshold and also stays stable within very close range to it for most of the curve, while it is followed by A2C, DQN, REINFORCE and AC in that order, with A2C getting better rewards at times towards the end of the timesteps.

through epochs and the entropy bonus may have accelerated learning, while clipping the objective and KL Divergence Approximation helped maintain stability.

One of the main influence factors for these results is the selected parameters. As smaller values of kl and ϵ guaranteed smaller steps in the policy, which could explain the rapid yet stable learning. Even so, towards the end of the budget some instability is still present in the curve. To improve it further, future work could be implementing Generalized Advantage Estimation(GAE) as an advantage function, as the current implementation just uses the normalized advantage function used in A2C.

The hyperparameter tuning process even though effective, was not as thorough as it could have been. To find the best possible parameters a wider selection of possible parameter values should be added into each of the tests. Besides this, other relevant variables like the rollout batch size, number of hidden layers in the network and learning rate could have also been tested to see how these impact the final results, but time constraints and resources limited this testing. For future work, this would be the main change besides the implementation of GAE, expanding upon the hyperparameter testing process.

5. Conclusion

The Proximal Policy Optimization algorithm was implemented and evaluated in the Cartpole-v1 environment, the final results after hyperparameter testing were compared with the results of previous implementations of Q-Learning, REINFORCE, Actor Critic and Advantage Actor Critic. The experiments showed how PPO achieves both fast and stable learning when compared to the other algorithms, being both the fastest in reaching the max reward and the one showing the most stable performance throughout the experiments.

References

Kakade, S. and Langford, J. Approximately optimal approximate reinforcement learning. In *Proceedings of the Nineteenth International Conference on Machine Learning*, ICML '02, pp. 267–274, San Francisco, CA, USA, 2002. Morgan Kaufmann Publishers Inc. ISBN 1558608737.

Nanda, A. Proximal policy optimization with pytorch and gymnasium, November 2024. URL https://www.datacamp.com/tutorial/proximal-policy-optimization. Accessed: 2026-05-18.

Schulman, J. Approximating KL divergence. http://joschu.net/blog/kl-approx.html, 2020. Accessed: 2026-05-18.

Schulman, J., Levine, S., Moritz, P., Jordan, M., and Abbeel, P. Trust region policy optimization. 02 2015.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *ArXiv*, abs/1707.06347, 2017. URL https://api.semanticscholar.org/CorpusID:28695052.

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

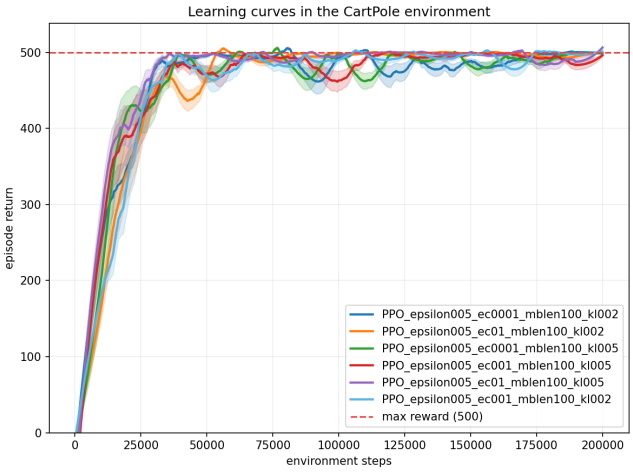
Yu, E. Y. Coding ppo from scratch with pytorch (part 4/4). https://medium.com/@z4xia/4e21f4a63e5c, 2023. Medium article, accessed 2026-05-18.

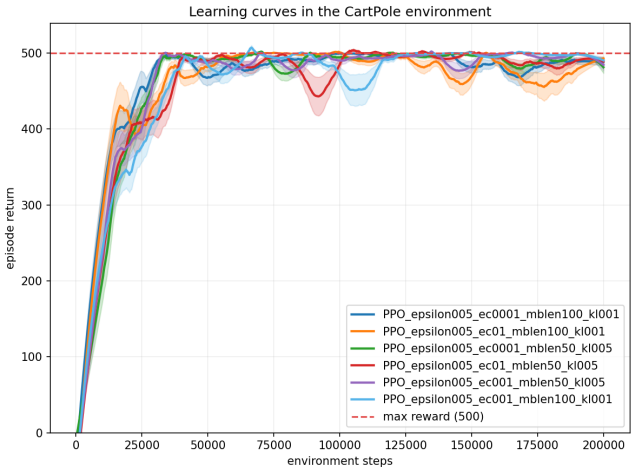
Appendix

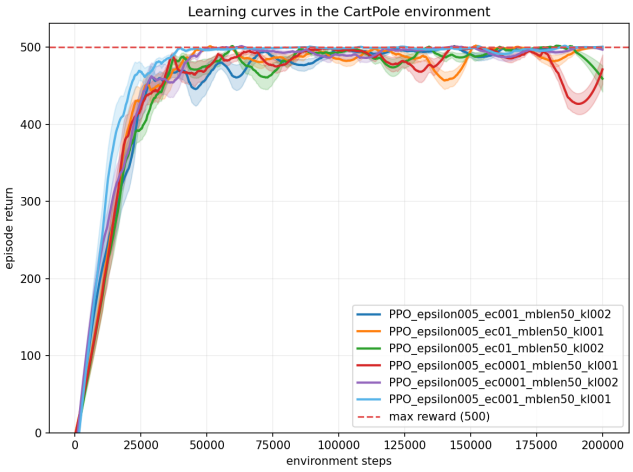
A. Hyperparameter Testing

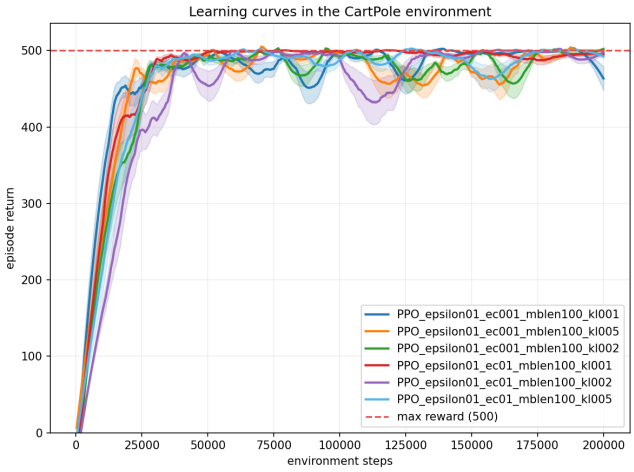
Parameter	Options during testing	Final run
Minibatch size B	[50,100]	50
Clipping parameter ϵ	[0.05,0.1,0.2]	0.05
Entropy coefficient c	[0.1,0.01,0.001]	0.01
Target Approximated KL Divergence kl	[0.01,0.02,0.05]	0.01
Actor learning rate α_θ	10^{-4}	10^{-4}
Critic learning rate α_ϕ	10^{-4}	10^{-4}
Number of epochs M	4	4
Discount factor γ	0.99	0.99
Number of steps N	2×10^5	10^6
Rollout batch size	2000	2000
Number of hidden layers	2	2
Number of repetitions	2	5

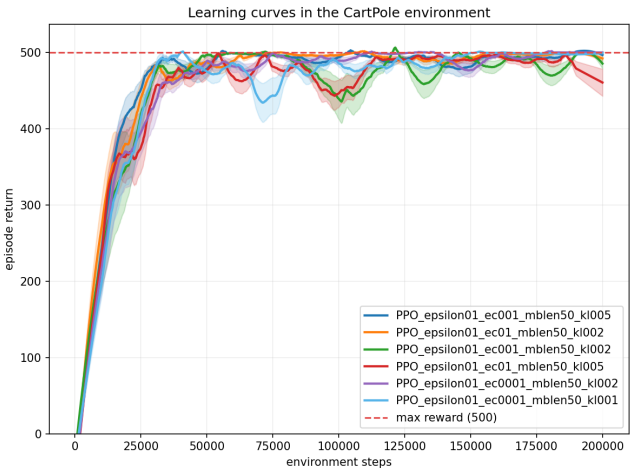
Table 2. Options for the parameters used during the PPO hyperparameter tuning process. Only the 4 first parameters were optimized, and the budget N and number of repetitions averaged was reduced in comparison to the final experiment run

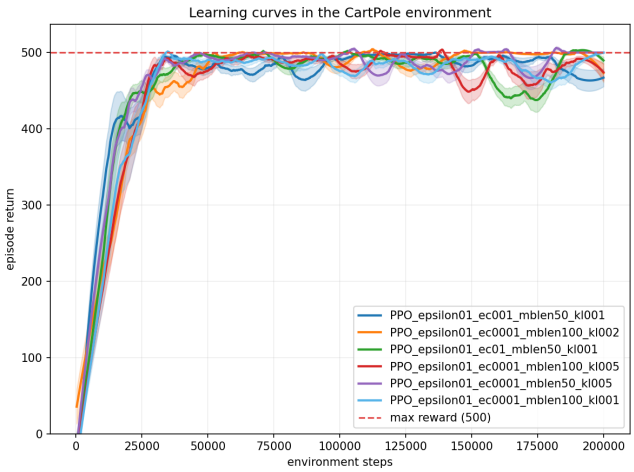

Learning curves in the CartPole environment
episode return
environment steps
PPO_epsilon005_ec0001_mblen100_kl002
PPO_epsilon005_ec01_mblen100_kl002
PPO_epsilon005_ec0001_mblen100_kl005
PPO_epsilon005_ec001_mblen100_kl005
PPO_epsilon005_ec01_mblen100_kl005
PPO_epsilon005_ec001_mblen100_kl002
max reward (500)


Learning curves in the CartPole environment
episode return
environment steps
PPO_epsilon005_ec0001_mblen100_kl001
PPO_epsilon005_ec01_mblen100_kl001
PPO_epsilon005_ec0001_mblen50_kl005
PPO_epsilon005_ec01_mblen50_kl005
PPO_epsilon005_ec001_mblen50_kl005
PPO_epsilon005_ec001_mblen100_kl001
max reward (500)


Learning curves in the CartPole environment
episode return
environment steps
PPO_epsilon005_ec001_mblen50_kl002
PPO_epsilon005_ec01_mblen50_kl001
PPO_epsilon005_ec01_mblen50_kl002
PPO_epsilon005_ec0001_mblen50_kl001
PPO_epsilon005_ec0001_mblen50_kl002
PPO_epsilon005_ec001_mblen50_kl001
max reward (500)


Learning curves in the CartPole environment
episode return
environment steps
PPO_epsilon01_ec001_mblen100_kl001
PPO_epsilon01_ec001_mblen100_kl005
PPO_epsilon01_ec001_mblen100_kl002
PPO_epsilon01_ec01_mblen100_kl001
PPO_epsilon01_ec01_mblen100_kl002
PPO_epsilon01_ec01_mblen100_kl005
max reward (500)


Learning curves in the CartPole environment
episode return
environment steps
PPO_epsilon01_ec001_mblen50_kl005
PPO_epsilon01_ec01_mblen50_kl002
PPO_epsilon01_ec001_mblen50_kl002
PPO_epsilon01_ec01_mblen50_kl005
PPO_epsilon01_ec0001_mblen50_kl002
PPO_epsilon01_ec0001_mblen50_kl001
max reward (500)


Learning curves in the CartPole environment
episode return
environment steps
PPO_epsilon01_ec001_mblen50_kl001
PPO_epsilon01_ec0001_mblen100_kl002
PPO_epsilon01_ec01_mblen50_kl001
PPO_epsilon01_ec0001_mblen100_kl005
PPO_epsilon01_ec0001_mblen50_kl005
PPO_epsilon01_ec0001_mblen100_kl001
max reward (500)

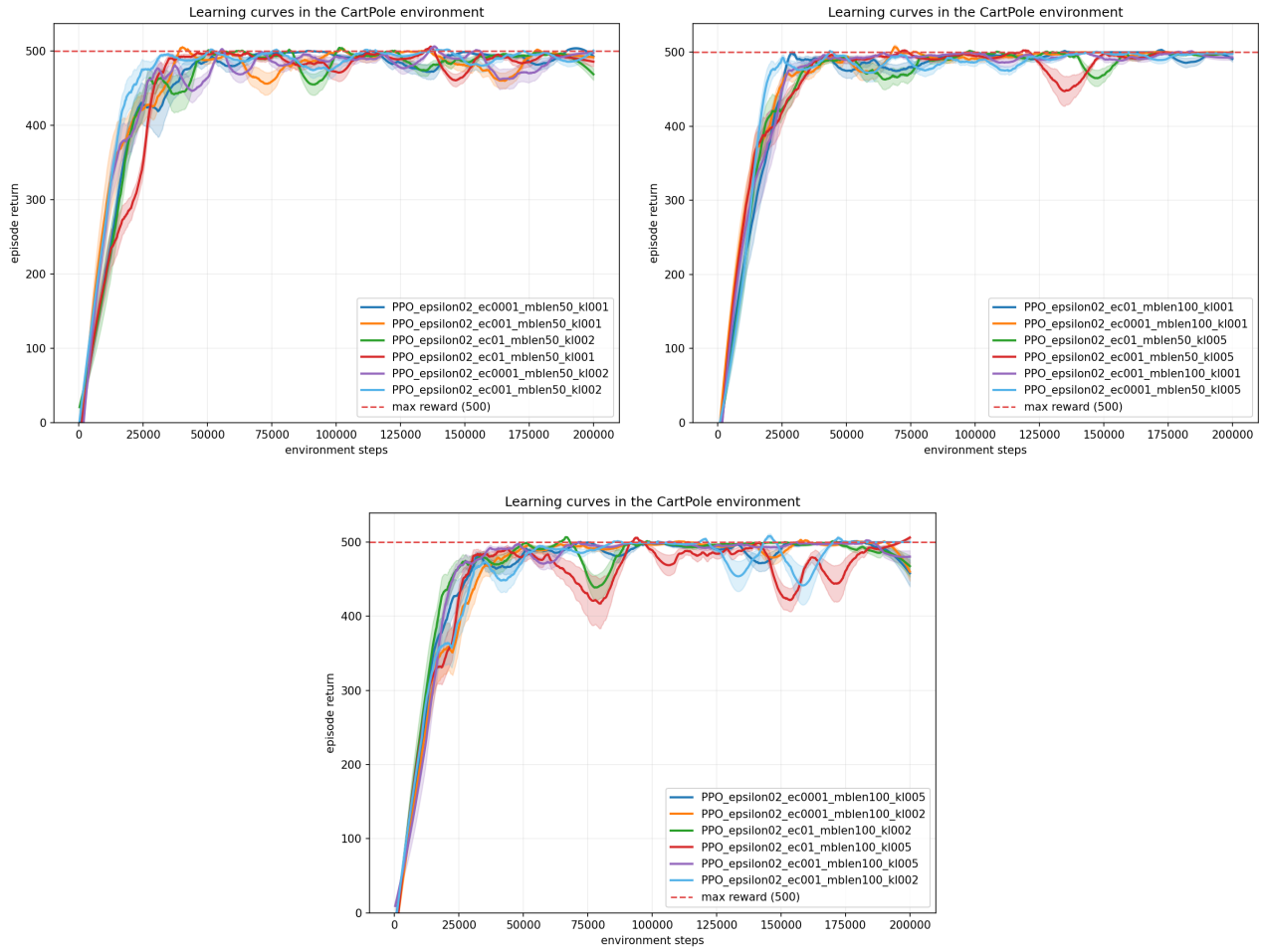


Figure 2. 9 figures for Comparison of the learning curves for PPO using different parameter settings, using non-normalized advantage. Results were averaged over 2 repetitions. Savitzky–Golay filter was applied to the data with a smoothing window of 81. All of the settings, regardless of which parameter values were selected, reached the max reward possible at least once. Results with higher kl values seemed to exhibit worse stability towards the end, same for higher epsilon values. The entropy coefficient c and the minibatch size B showed less of an impact within the 2×10^5 budget, . Given these results, a few candidates were selected for further comparison

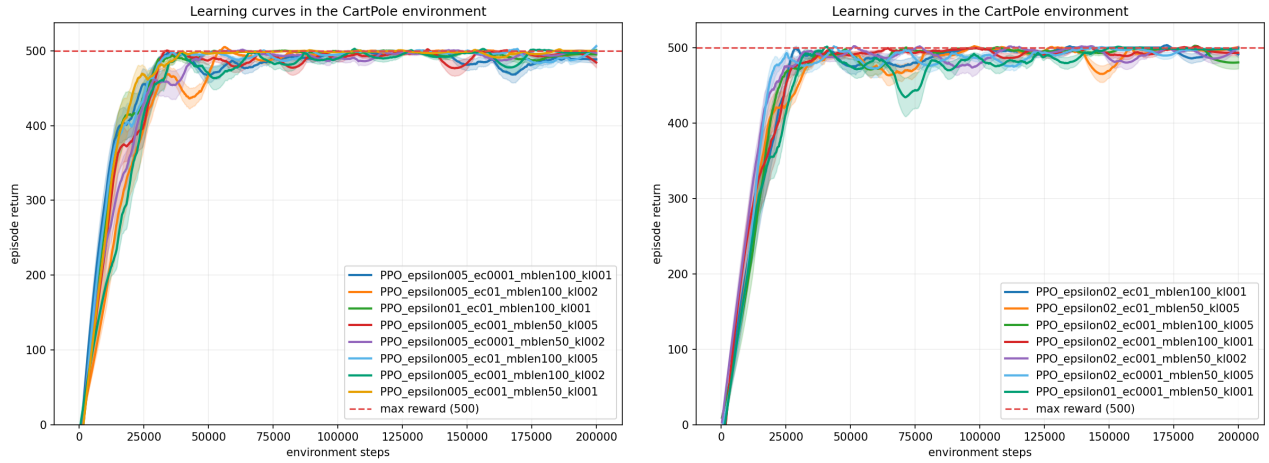


Figure 3. Comparison of the selected curves from the previous comparisons. Final parameter candidates were selected from this comparison, selecting 2 combinations for every value of ϵ for another test, now ran using the same budget and repetitions as the final experiment. Even so, from this stage, curves with an ϵ value of 0.05 seemed to be the most stable

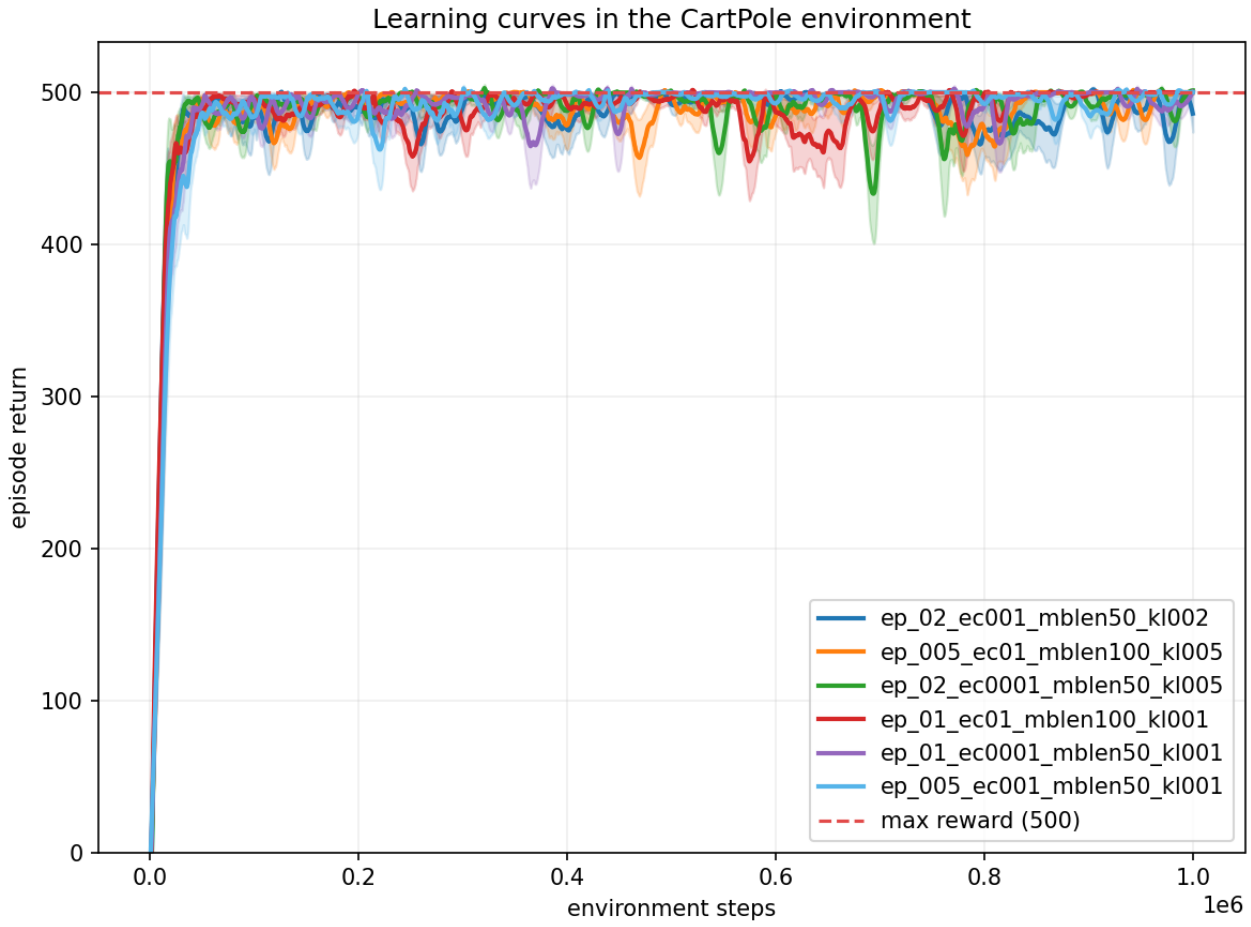


Figure 4. Final parameter setting comparison using 6 candidates. Curves that used higher ϵ and kl values reached maximum reward the quickest, but showed more signs of instability and possible points of collapse during the experiment. The candidate curve with the values $\epsilon = 0.05$, $c = 0.01$, $B = 50$ and $kl = 0.01$ was selected for the final run, as even though slower than other curves when reaching the peak,it stayed at it consistently when compared to the others.